

Resource Allocation System

Iteration 34 – Committee Meeting Request Model

Najib Attar
May 2026

1 Iteration Goal

The goal of Iteration 34 is to add a new request type called `CommitteeMeetingRequest`. This request type represents academic meetings such as thesis presentations, project presentations, jury meetings, and committee meetings.

Earlier iterations added room allocation, exam allocation, user busy slots, and user availability services. However, the system still could not represent a request that requires multiple human participants. This iteration adds the request model and participant mapping foundation for future multi-person scheduling.

This iteration does not implement participant availability checking, automatic time suggestions, or optimization.

2 Changes from Previous Iteration

Iteration 33 introduced the user availability foundation using `UserBusySlot` and `UserAvailabilityService`. Iteration 34 builds on that foundation by introducing a request type that can store required participants.

The main changes are:

- Added `CommitteeMeetingRequest`.
- Added `RequestParticipant`.
- Added `data/request_participants.csv`.
- Updated `RequestFactory` to support `requestType = CommitteeMeeting`.
- Updated `DataLoader` to load participant mappings.
- Updated `DataController` and `SystemData` to store and attach participants.
- Updated rules, strategies, service, and output components so committee meeting requests can be processed safely.
- Updated user subclasses so `CommitteeMeeting` is an allowed request type.
- Added sample `CommitteeMeeting` request 40.
- No web files were modified.

Existing request types and allocation strategies remain compatible.

3 Scenario

The professor introduced a scenario where a meeting or presentation requires multiple professors at the same time. For example, a master student may want to present their work. The presentation may require:

- a supervisor,
- one committee member,
- another committee member.

The system should eventually check whether all required professors and a suitable room are available at the requested time. Before this iteration, the system could represent room-based requests and exam requests, but it could not represent required human participants.

4 Modules and Their Tasks

Module	Task
<code>CommitteeMeetingRequest</code>	Stores meeting-specific request data such as topic, duration, preferred time slot, and required participant IDs.
<code>RequestParticipant</code>	Stores participant mapping data between a request and a required user.
<code>RequestFactory</code>	Creates <code>CommitteeMeetingRequest</code> when <code>requestType</code> is <code>CommitteeMeeting</code> .
<code>DataLoader</code>	Loads participant mappings from <code>request_participants.csv</code> .
<code>DataController</code>	Attaches participant mappings to matching committee meeting requests.
<code>SystemData</code>	Stores loaded request participants with users, spaces, requests, and busy slots.
Rules and Strategies	Process <code>CommitteeMeetingRequest</code> like a one-time room request for now.

Table 1: Main modules involved in Iteration 34

5 Committee Meeting Request Design

`CommitteeMeetingRequest` inherits from the abstract `Request` class. It represents a one-time meeting in schedule, but it is not modeled only as `OneTimeRequest` because it has additional meeting-specific meaning and required participants.

The main fields are:

- `meetingTopic`

- durationMinutes
- preferredTimeSlot
- requiredParticipantIds
- participantRoles

The important methods are:

- getMeetingTopic()
- getDurationMinutes()
- getPreferredTimeSlot()
- getRequiredParticipantIds()
- getParticipantRoles()
- addRequiredParticipant(...)
- isRecurring()
- getRequestType()

For now, `CommitteeMeetingRequest` is processed like a one-time request for room allocation. It uses the normal rule pipeline:

- RequestTypeRule
- UserRoleRule
- CapacityRule
- FeatureRule
- StatusRule
- LocationRule
- AvailabilityRule

Participant availability is intentionally not checked yet.

6 Participant Mapping Design

A separate `RequestParticipant` model was added to store required participant mappings. This keeps many-to-many participant data separate from the main request row.

The new CSV file is:

`data/request_participants.csv`

Its format is:

`requestId,userId,participantRole`

Field	Meaning
<code>requestId</code>	The committee meeting request ID.
<code>userId</code>	The required participant user ID.
<code>participantRole</code>	The participant's role in the meeting.

Table 2: Fields in request participant mapping

Example participant mappings are:

`40,2,Supervisor`
`40,4,CommitteeMember`
`40,6,CommitteeMember`

This design is better than adding fixed columns such as `professor1`, `professor2`, and `professor3` to `requests.csv`. Different meetings may require different numbers of participants, so a separate mapping table is more flexible and extensible.

7 CSV Design

The main `requests.csv` format remains unchanged:

`requestId,requestType,userId,spaceId,participantCount,`
`requiredFeature,requiredBuilding,timeData,title,purpose,`
`courseCode,courseName,examType,canSplitAcrossRooms`

For a committee meeting request:

- `requestType = CommitteeMeeting`
- `title` describes the meeting topic
- `purpose` can be `Presentation` or `CommitteeMeeting`
- `timeData` stores the preferred meeting time

- `participantCount` represents expected room attendee count
- exam-specific fields are unused

The sample request added in this iteration is:

```
40,CommitteeMeeting,1,301,10,Projector,AdminBuilding,
5-13:00-14:00,Master Thesis Presentation,Presentation,,,,false
```

The participant users are stored separately in `request_participants.csv`.

8 Request-Type Permission

`CommitteeMeeting` is allowed for all current user subclasses:

- `Student`
- `TeachingAssistant`
- `Staff`
- `Instructor`
- `Administrator`

This policy supports the academic scenario. A student may request a thesis presentation meeting. Teaching assistants or staff may coordinate academic meetings. Instructors and administrators can also schedule committee meetings.

Exam requests remain separate and are still restricted to `Instructor` and `Administrator`.

9 Current Limitation and Next Step

This iteration does not yet check whether the required professors are available.

The next iteration should add `ParticipantAvailabilityRule`. That rule should use `UserAvailabilityService`, `user_busy_slots.csv`, and the required participant IDs stored inside `CommitteeMeetingRequest`. It should reject committee meetings when one or more required participants are busy at the preferred time.

This iteration also does not implement automatic time suggestions, least-change scheduling, or meeting optimization.

10 Iteration Comparison

Aspect	Before Iteration 34	After Iteration 34
Committee meeting request type	Not supported	Supported through <code>CommitteeMeetingRequest</code>
Required participants	Not represented	Stored through <code>RequestParticipant</code> mappings
Main request CSV schema	Existing schema	Unchanged
Participant CSV	Not available	Added <code>request_participants.csv</code>
Participant availability checking	Not implemented	Still not implemented
Room allocation behavior	Existing request types only	Committee meeting processed like one-time room request
Existing strategies	Already supported	Still supported
Web forms	Existing web dashboard	No web changes in this iteration

Table 3: Comparison before and after Iteration 34

11 Updated Class Diagram

Figure 1 shows the updated class diagram for this iteration. The diagram should include `Request`, `CommitteeMeetingRequest`, `RequestParticipant`, `RequestFactory`, `DataLoader`, `DataController`, `SystemData`, and `TimeSlot`.

The important relationships are:

- `CommitteeMeetingRequest` inherits from `Request`.
- `CommitteeMeetingRequest` stores required participant IDs and roles.
- `RequestParticipant` maps `requestId` to `userId` and `participantRole`.
- `DataLoader` loads `RequestParticipant` objects.
- `SystemData` stores `requestParticipants`.
- `DataController` attaches participant mappings to `CommitteeMeetingRequest`.

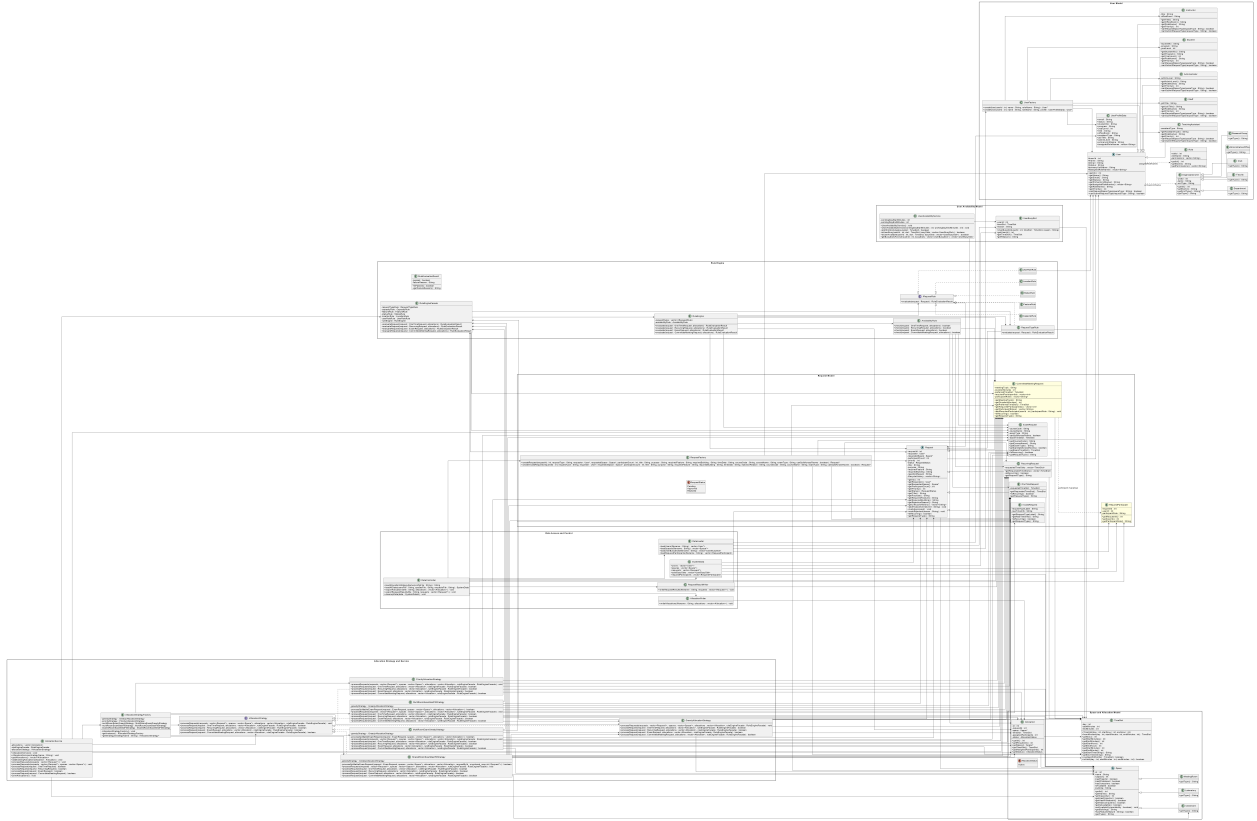


Figure 1: Updated class diagram after Iteration 34

12 Updated Workflow Diagram

Figure 2 shows the updated workflow for this iteration. The workflow now includes loading `request_participants.csv`, creating committee meeting requests through `RequestFactory`, and attaching participant mappings through `DataController`.

For now, the committee meeting request is evaluated like a one-time room request. A future workflow will add participant availability checking.

one-time slot, and the duration is computed from the preferred time slot.

DataLoader loads participant mappings from `data/request_participants.csv`. If the file is missing, the system prints a warning and continues with an empty participant list. If a participant row is malformed, it is skipped safely.

DataController stores the participant mappings in **SystemData** and attaches matching participants to each **CommitteeMeetingRequest**. If no participants are found for a committee meeting request, the request is still kept valid for now. Participant validation is left for a future rule.

The sample request 40 was loaded as a **CommitteeMeetingRequest**. It received three participant mappings and was approved and allocated to room M301 on Friday 13:00–14:00 when the normal room-based rules passed.

14 Summary

Iteration 34 adds the request-type foundation for future multi-person meeting scheduling. The new **CommitteeMeetingRequest** represents academic meetings and presentations that require multiple users. The new **RequestParticipant** model stores participant mappings separately from the main request CSV file.

The design preserves the existing architecture. **CommitteeMeetingRequest** stores meeting-specific data, **RequestParticipant** stores mapping data, **DataLoader** loads CSV data, and **DataController** attaches participants. Existing request types, rules, and allocation strategies remain compatible.

The next step is to add **ParticipantAvailabilityRule**, which will use **UserAvailabilityService** and `user_busy_slots.csv` to reject committee meeting requests when required participants are unavailable.